Module 2

Process and threads

Dr. Ngangbam Indrason
Assistant Professor - SCOPE
VIT-AP University, Amaravati

Contents

- Process and programs
- Process states
- Process concept
- Process scheduling
- Operations on processes
- Interprocess Communication (IPC)
- Concurrency
- Interacting processes
- Multithreading models

Process and programs

- **Program**: A passive set of instructions stored on disk (e.g., .exe, .out, .sh files).
- **Process**: An active instance of a program in execution. It includes:
  - Program Counter (PC)
  - Stack (function calls, parameters)
  - Heap (dynamically allocated memory)
  - Data section (global/static variables)
  - Registers

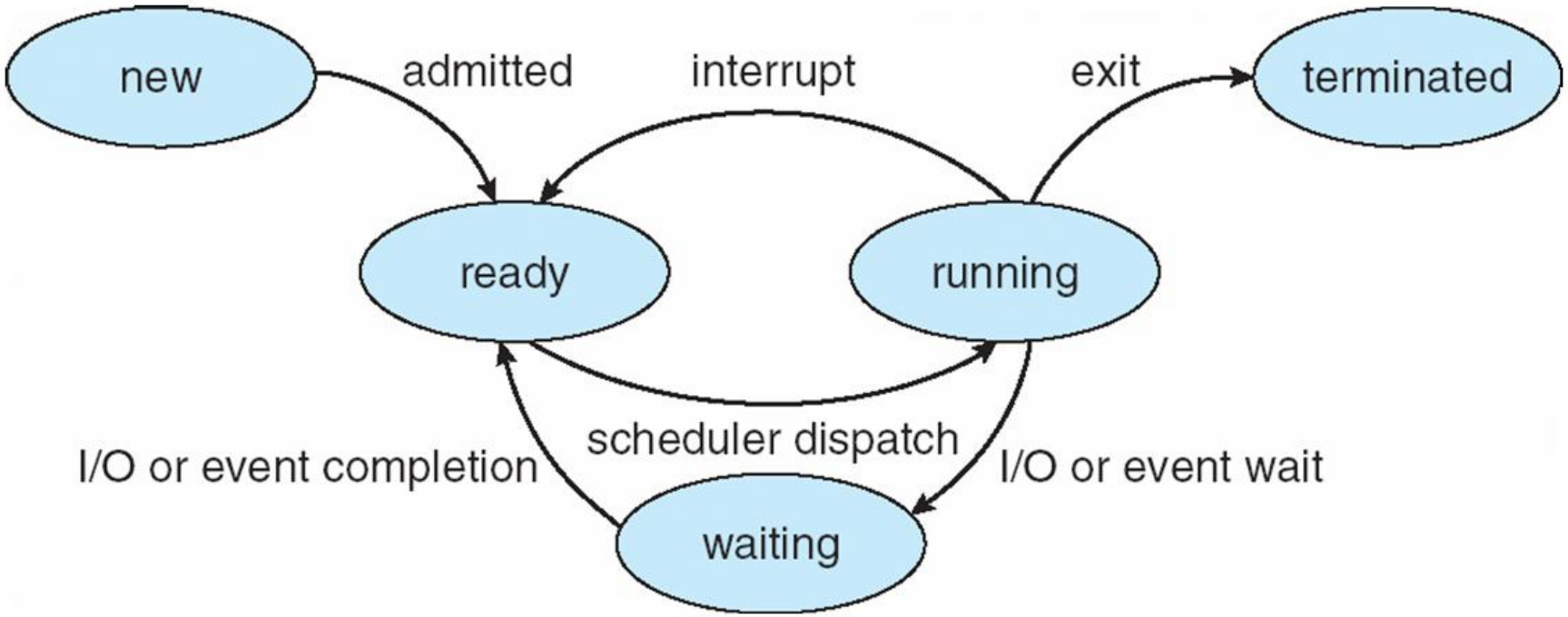
Process and programs

Aspect	Program	Process
Nature	Passive (code)	Active (execution)
Stored in	Disk	Memory (RAM)
Lifecycle	Static	Dynamic

Process states

- A process undergoes multiple states during its lifetime:
  - **new**: The process is being created
  - **ready**: The process is waiting to be assigned to a processor
  - **running**: Instructions are being executed
  - **waiting**: The process is waiting for some event to occur
  - **terminated**: The process has finished execution

Process states

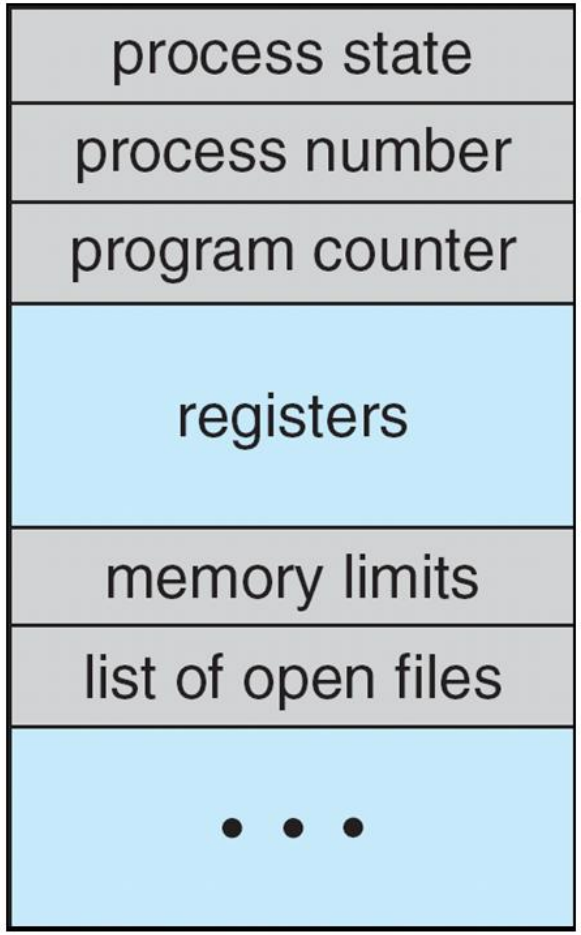


Process concept

- Components of a Process:
  - **Program Counter**: Tracks next instruction.
  - **Registers**: Store intermediate data.
  - **Stack**: Holds function calls and local variables.
  - **Heap**: Dynamic memory allocation.
  - **Data Section**: Global variables.
- **Process Control Block (PCB)**: OS uses PCB to manage processes.
- **Context Switching**: The process of saving the current state of a CPU and loading another process's state. It allows multitasking.

Process concept

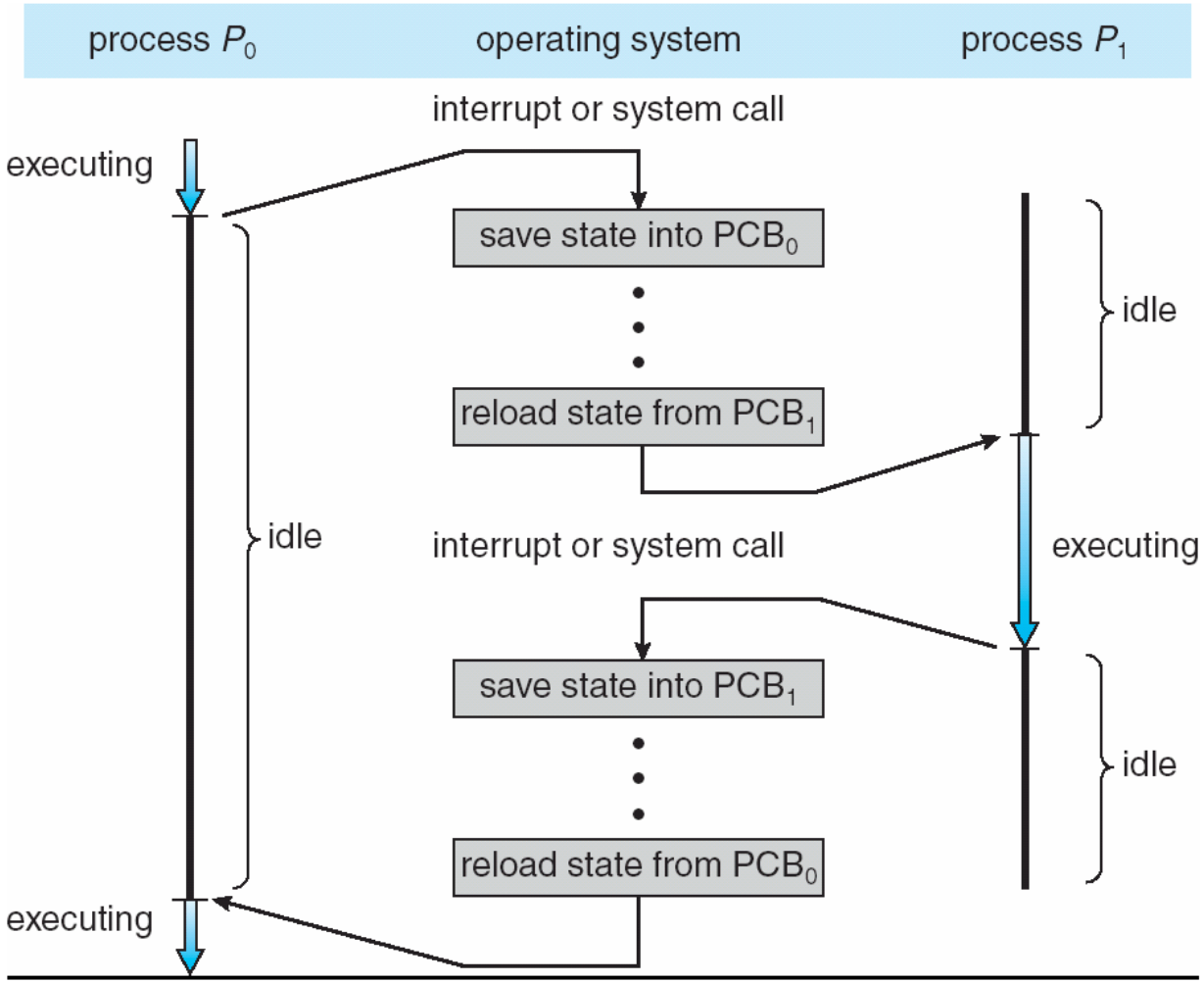
- **Process Control Block (PCB)**
  - *Process state* – running, waiting, etc
  - *Program counter* – location of instruction to next execute
  - *CPU registers* – contents of all process-centric registers
  - *CPU scheduling information*- priorities, scheduling queue pointers
  - *Memory-management information* – memory allocated to the process
  - *Accounting information* – CPU used, clock time elapsed since start, time limits
  - *I/O status information* – I/O devices allocated to process, list of open files



Context Switch

- When CPU switches to another process, the system must save the state of the old process and load the saved state for the new process via a context switch.
- Context of a process represented in the PCB.
- Context-switch time is overhead; the system does no useful work while switching
  - The more complex the OS and the PCB → the longer the context switch
- Time dependent on hardware support
  - Some hardware provides multiple sets of registers per CPU → multiple contexts loaded at once

CPU Switch From Process to Process



Process scheduling

- Maximize CPU use, quickly switch processes onto CPU for time sharing.
- Process scheduler selects among available processes for next execution on CPU.
- Maintains scheduling queues of processes:
  - *Job queue* – set of all processes in the system
  - *Ready queue* – set of all processes residing in main memory, ready and waiting to execute
  - *Device queues* – set of processes waiting for an I/O device
  - Processes migrate among the various queues

Process scheduling

- **Scheduling Criteria**:
  - *CPU Utilization*: Amount of time the CPU is actively working
  - *Throughput*: The number of processes completed per unit time.
  - *Turnaround Time*: The total time taken from process submission to its completion.
  - *Waiting Time*: The total time a process spends in the ready queue (waiting for CPU).
  - *Response Time*: Time from process submission to the first time it gets the CPU.
- **Types of Schedulers**:
  - *Long-Term Scheduler (Job Scheduler)*: Selects processes from job pool and loads them into memory.
  - *Short-Term Scheduler (CPU Scheduler)*: Selects from ready queue for execution.
  - *Medium-Term Scheduler*: Swaps out processes to reduce memory load.

Process scheduling

- Scheduling Algorithms:
  - FCFS (First Come First Serve)
  - SJF (Shortest Job First)
  - Round Robin
  - Priority Scheduling
  - Multilevel Queue Scheduling
- Objectives of Process Scheduling:
  - Max CPU utilization, Max throughput, Min turnaround time, Min waiting time, Min response time

First- Come, First-Served (FCFS) Scheduling

Process	Burst Time
P1	24
P2	3
P3	3

- Suppose that the processes arrive in the order: P1 , P2 , P3
  The Gantt Chart for the schedule is:

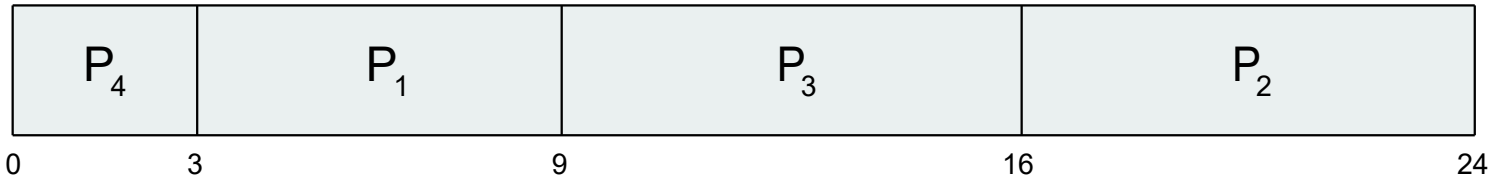
- Waiting time for P1 = 0; P2 = 24; P3 = 27
- Average waiting time: (0 + 24 + 27)/3 = 17

Shortest-Job-First (SJF) Scheduling

- SJF is optimal – gives minimum average waiting time for a given set of processes

Process	Burst Time
P1	6
P2	8
P3	7
P4	3

- SJF scheduling chart



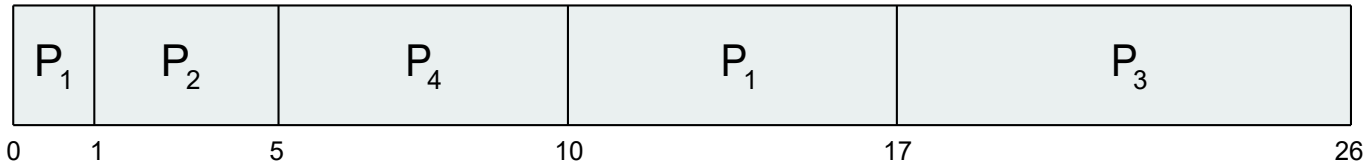
- Average waiting time = (3 + 16 + 9 + 0) / 4 = 7

Shortest-remaining-time-first

- We add the concepts of varying arrival times and preemption to the analysis.

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

- Preemptive SJF Gantt Chart



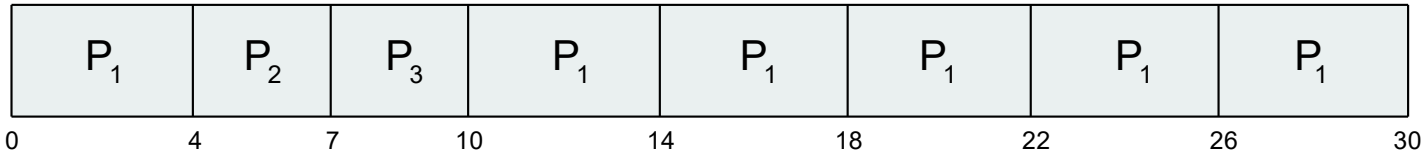
- Average waiting time = [(10-1)+(1-1)+(17-2)+5-3)]/4 = 26/4 = 6.5 msec

Round Robin (RR)

- Example of RR with Time Quantum (q) = 4

Process	Burst Time
P1	24
P2	3
P3	3

- The Gantt chart is:



- Typically, higher average turnaround than SJF, but better response
- q should be large compared to context switch time
- q usually 10ms to 100ms, context switch < 10 µsec

Priority Scheduling

- A priority number (integer) is associated with each process.

Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

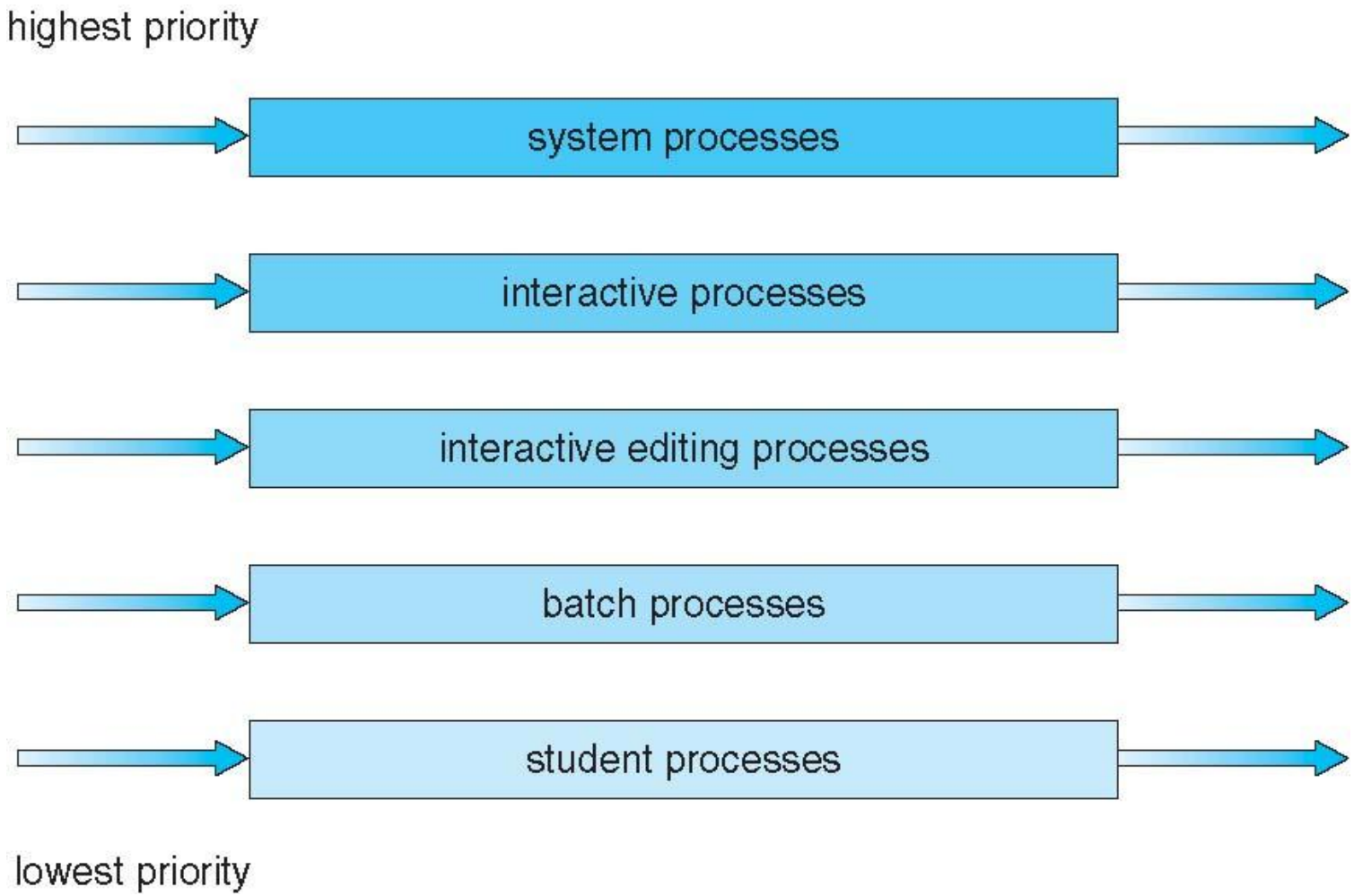
- Priority scheduling Gantt Chart

- Average waiting time = 8.2 msec

Multilevel Queue Scheduling

- Ready queue is partitioned into separate queues, eg, *foreground (interactive)* and *background (batch).*
- Each process is permanently assigned to one specific queue, based on its type or characteristics.
- Each queue has its own scheduling algorithm: *foreground – RR* and *background – FCFS.*
- Scheduling must be done between the queues:
  - Fixed priority scheduling; (i.e., serve all from foreground then from background). Possibility of starvation.
  - Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR
  - 20% to background in FCFS

Multilevel Queue Scheduling



Process scheduling

Algorithm	Description	Use Case
FCFS	First Come First Serve	Simple batch systems
SJF	Shortest Job First	Minimizes average wait time
Round Robin	Time-slice based	Time-sharing systems
Priority	Based on priority value	Real-time systems
Multilevel Queue	Multiple queues for different types	Complex systems

Operations on processes

- **Process Creation:**
  - Parent creates a child process via system calls like *fork()*, *exec()*.
  - Child can: Run concurrently with parent, Be a duplicate (copy of parent), Load a different program.
- **Process Termination:**
  - Process finishes execution or is aborted.
  - Use *exit()* system call.
  - Parent may use *wait()* to wait for child termination.
- **Other Operations:**
  - Suspend/resume
  - Inter-process communication setup

To be continued …